

NUnit Hands-On – Answers

1. Source Project – CollectionsLib

Employee.cs – with Equals and GetHashCode overridden

CollectionsLib/Employee.cs

```
using System;

namespace CollectionsLib { public class
Employee { public int Id { get; set; }
public string Name { get; set; } public
string Department { get; set; } public
int YearJoined { get; set; }

// Required for Scenario 3 (unique check) and Scenario 4 (AreEqual)
// Two employees are equal if their Id is the same public override
bool Equals(object obj) { if (obj == null || GetType() !=
obj.GetType()) return false; Employee other = (Employee)obj; return
this.Id == other.Id; }

public override int GetHashCode() {
return Id.GetHashCode(); }
}
}
```

CollectionsManager.cs – two collection methods

CollectionsLib/CollectionsManager.cs

```
using System; using
System.Collections.Generic;

namespace CollectionsLib { public class CollectionsManager { // Returns
a list of employees. Must include employee with Id = 100. // All Ids
must be unique.
public ICollection GetEmployees()
```

```

{ return new List { new Employee { Id=100, Name="Alice", Department="IT",
YearJoined=2022 }, new Employee { Id=101, Name="Bob", Department="HR",
YearJoined=2021 }, new Employee { Id=102, Name="Charlie", Department="IT",
YearJoined=2020 }, new Employee { Id=103, Name="Diana", Department="Finance",
YearJoined=2019 } };
}

// Returns employees who joined before current year. // Must contain
the same employees as GetEmployees() for Scenario 4.
public ICollection GetEmployeesWhoJoinedInPreviousYears() { return new List {
new Employee { Id=100, Name="Alice", Department="IT", YearJoined=2022 }, new
Employee { Id=101, Name="Bob", Department="HR", YearJoined=2021 }, new Employee
{ Id=102, Name="Charlie", Department="IT", YearJoined=2020 }, new Employee {
Id=103, Name="Diana", Department="Finance", YearJoined=2019 } };
}
}
}

```

2. Test Project Setup – CollectionsLib.Tests

- In the same solution: File → Add → New Project → Class Library (C#).
- Name it CollectionsLib.Tests.
- Rename the default class file to CollectionsManagerTests.cs.
- Add a project reference to CollectionsLib.

Install NuGet packages

Package Manager Console

```

Install-Package NUnit
Install-Package NUnit3TestAdapter
Install-Package Microsoft.NET.Test.Sdk

```

Test class skeleton

CollectionsLib.Tests/CollectionsManagerTests.cs – skeleton

```

using
System.Collections.Generic;
using NUnit.Framework; using
CollectionsLib;

namespace CollectionsLib.Tests {
[TestFixture] public class
CollectionsManagerTests { private
CollectionsManager _manager;
private ICollection _employees;

[SetUp] public
void Init() {
_manager = new CollectionsManager();
_employees = _manager.GetEmployees();
}

// --- test methods go here ---
}
}

```

3. Scenario 1 – Collection Contains No Null Values

Use `CollectionAssert.AllItemsAreNotNull` to verify that none of the `Employee` objects in the collection is null. Both Classic Model and Constraint Model versions are shown.

Classic Model (`CollectionAssert`)

Scenario 1 – Classic Model

```

// Classic Model [Test] public void
GetEmployees_Collection_AllItemsAreNotNull() {
// Assert: no item in the collection is null
CollectionAssert.AllItemsAreNotNull(_employees);
}

```

Constraint Model (`Assert.That`)

Scenario 1 – Constraint Model

```

// Constraint Model [Test] public void
GetEmployees_Collection_AllItemsAreNotNull_Constraint() {
Assert.That(_employees, Is.All.Not.Null);
}

```

[Screenshot – Test Explorer: Scenario 1 Passed]

```

Test Explorer - Scenario 1
Passed GetEmployees_Collection_AllItemsAreNotNull 5 ms Passed
GetEmployees_Collection_AllItemsAreNotNull_Constraint 4 ms
2 / 2 passed

```

4. Scenario 2 – Employee with Id 100 Exists in Collection

Create an Employee object with Id = 100 and use CollectionAssert.Contains to verify it is in the collection. Because we overrode Equals() to compare by Id, NUnit can find the match.

Classic Model

Scenario 2 – Classic Model

```
// Classic Model [Test] public void
GetEmployees_Collection_ContainsEmployeeWithId100() { // Create an
employee object with Id = 100 to search for var empToFind = new
Employee { Id = 100 };

// Equals() compares by Id so this will find the match
CollectionAssert.Contains(_employees, empToFind);
}
```

Constraint Model

Scenario 2 – Constraint Model

```
// Constraint Model [Test] public void
GetEmployees_Collection_ContainsEmployeeWithId100_Constraint() {
var empToFind = new Employee { Id = 100 };

Assert.That(_employees, Does.Contain(empToFind));
}
```

[Screenshot – Test Explorer: Scenario 2 Passed]

```
Test Explorer - Scenario 2
Passed GetEmployees_Collection_ContainsEmployeeWithId100 6 ms Passed
GetEmployees_Collection_ContainsEmployeeWithId100_Constraint 5 ms
2 / 2 passed
```

5. Scenario 3 – All Employees Are Unique (No Duplicate Ids)

Use CollectionAssert.AllItemsAreUnique to check that no two Employee objects in the list share the same Id. NUnit uses Equals() to determine uniqueness – which is why we overrode it to compare by Id.

Classic Model

Scenario 3 – Classic Model

```
// Classic Model [Test] public void
GetEmployees_Collection_AllItemsAreUnique() { // Fails if any two
employees have the same Id (Equals() returns true)
CollectionAssert.AllItemsAreUnique(_employees);
}
```

Constraint Model

Scenario 3 – Constraint Model

```
// Constraint Model [Test] public void
GetEmployees_Collection_AllItemsAreUnique_Constraint() {
Assert.That(_employees, Is.Unique);
}
```

[Screenshot – Test Explorer: Scenario 3 Passed]

```
Test Explorer - Scenario 3
```

```
Passed GetEmployees_Collection_AllItemsAreUnique 5 ms Passed
GetEmployees_Collection_AllItemsAreUnique_Constraint 4 ms
2 / 2 passed
```

Breaking the test – adding a duplicate

To observe a failing test we add a duplicate Id in CollectionsManager and re-run:

CollectionsManager.cs – deliberately broken (duplicate Id=100)

```
// Modified GetEmployees() to introduce a duplicate return new List { new
Employee { Id=100, Name="Alice", Department="IT" }, new Employee {
Id=100, Name="Alice2", Department="IT" }, // <-- duplicate new Employee {
Id=101, Name="Bob", Department="HR" }, new Employee { Id=102,
Name="Charlie", Department="IT" } };
```

[Screenshot – Test Explorer: Scenario 3 FAILED (duplicate introduced)]

```
Test Explorer - Scenario 3 (broken)

Failed GetEmployees_Collection_AllItemsAreUnique
Message: Expected: all items unique
But was: duplicate found at index 1

Failed GetEmployees_Collection_AllItemsAreUnique_Constraint
Message: Expected: unique

But was: < Employee{Id=100}, Employee{Id=100}, ... >

0 / 2 passed
```

6. Scenario 4 – Both Collections Contain the Same Items

GetEmployees() and GetEmployeesWhoJoinedInPreviousYears() both return Employee collections. We verify they contain exactly the same items.

- AreEqual – checks same items in the same order.
- AreEquivalent – checks same items regardless of order.

Classic Model – AreEqual (same order)

Scenario 4 – Classic Model AreEqual

```
// Classic Model - order matters [Test] public void
GetEmployees_AndPreviousYears_AreEqual() {
ICollection current = _manager.GetEmployees(); ICollection previous =
_manager.GetEmployeesWhoJoinedInPreviousYears();

// Same items, same order
CollectionAssert.AreEqual(current, previous);
}
```

Classic Model – AreEquivalent (any order)

Scenario 4 – Classic Model AreEquivalent

```
// Classic Model - order does not matter [Test] public
void GetEmployees_AndPreviousYears_AreEquivalent() {
    ICollection current = _manager.GetEmployees(); ICollection previous =
    _manager.GetEmployeesWhoJoinedInPreviousYears();

    // Same items, any order
    CollectionAssert.AreEqual(current, previous);
}
```

Constraint Model – AreEqual

Scenario 4 – Constraint Model

```
// Constraint Model [Test] public void
GetEmployees_AndPreviousYears_AreEqual_Constraint() {
    ICollection current = _manager.GetEmployees(); ICollection previous =
    _manager.GetEmployeesWhoJoinedInPreviousYears();

    // Is.EqualTo on collections checks count + element equality
    Assert.That(current, Is.EqualTo(previous));
}
```

[Screenshot – Test Explorer: Scenario 4 Passed]

```
Test Explorer - Scenario 4
Passed GetEmployees_AndPreviousYears_AreEqual 6 ms
Passed GetEmployees_AndPreviousYears_AreEquivalent 5 ms
Passed GetEmployees_AndPreviousYears_AreEqual_Constraint 5 ms
3 / 3 passed
```

Breaking the test – add an extra employee

Modify GetEmployeesWhoJoinedInPreviousYears() to return one extra employee and re-run to observe the failure:

CollectionsManager.cs – deliberately broken (extra employee)

```
public ICollection GetEmployeesWhoJoinedInPreviousYears() { return
new List { new Employee { Id=100, Name="Alice", Department="IT" },
new Employee { Id=101, Name="Bob", Department="HR" }, new Employee
{ Id=102, Name="Charlie", Department="IT" }, new Employee { Id=103,
Name="Diana", Department="Finance" }, new Employee { Id=104,
Name="Extra", Department="IT" } // <-- extra };
}
```

```
Test Explorer - Scenario 4 (broken)

Failed GetEmployees_AndPreviousYears_AreEqual Message:
Expected and actual collections differ in length.
Expected length: 4 But was: 5

Failed GetEmployees_AndPreviousYears_AreEquivalent
Message: Expected: equivalent to < ... 4 items ... >
But was: < ... 5 items ... >

Failed
GetEmployees_AndPreviousYears_AreEqual_Constraint
Message: Expected: < ... 4 items ... > But was: < ... 5
items ... >

0 / 3 passed
```

7. Complete Test File

```
using
System.Collections.Generic;
using NUnit.Framework; using
CollectionsLib;

namespace CollectionsLib.Tests {
[TestFixture] public class
CollectionsManagerTests { private
CollectionsManager _manager;
private ICollection _employees;

[SetUp] public
void Init() {
_manager = new CollectionsManager();
_employees = _manager.GetEmployees();
}

// ■■ Scenario 1: no null items ████████████████████████████████████████
[Test] public void
GetEmployees_Collection_AllItemsAreNotNull() {
CollectionAssert.AllItemsAreNotNull(_employees);
}

[Test] public void
GetEmployees_Collection_AllItemsAreNotNull_Constraint() {
Assert.That(_employees, Is.All.Not.Null);
}

// ■■ Scenario 2: employee id 100 exists ████████████████████████████████
[Test] public void
GetEmployees_Collection_ContainsEmployeeWithId100() { var
empToFind = new Employee { Id = 100 };
CollectionAssert.Contains(_employees, empToFind);
}

[Test] public void
GetEmployees_Collection_ContainsEmployeeWithId100_Constraint() { var
empToFind = new Employee { Id = 100 }; Assert.That(_employees,
Does.Contain(empToFind));
}

// ■■ Scenario 3: all items unique
```



```

[Test] public void
GetEmployees_Collection_AllItemsAreUnique() {
    CollectionAssert.AllItemsAreUnique(_employees);
}

[Test] public void
GetEmployees_Collection_AllItemsAreUnique_Constraint() {
    Assert.That(_employees, Is.Unique);
}

// ■■ Scenario 4: both collections equal
[Test] public void
GetEmployees_AndPreviousYears_AreEqual() {
    ICollection current = _manager.GetEmployees();
    ICollection previous =
        _manager.GetEmployeesWhoJoinedInPreviousYears();
    CollectionAssert.AreEqual(current, previous);
}

[Test] public void
GetEmployees_AndPreviousYears_AreEquivalent() {
    ICollection current = _manager.GetEmployees();
    ICollection previous =
        _manager.GetEmployeesWhoJoinedInPreviousYears();
    CollectionAssert.AreEqual(current, previous);
}

[Test] public void
GetEmployees_AndPreviousYears_AreEqual_Constraint() {
    ICollection current = _manager.GetEmployees();
    ICollection previous =
        _manager.GetEmployeesWhoJoinedInPreviousYears();
    Assert.That(current, Is.EqualTo(previous));
}
}
}
}

```

8. Final Test Run – All Tests

Click Run All in Test Explorer. All 9 tests should pass.

[Screenshot – Test Explorer: All 9 Tests Passed]

```

Test Explorer - Full Run
=====
CollectionsLib.Tests.CollectionsManagerTests

```

```
Passed GetEmployees_Collection_AllItemsAreNotNull 5 ms
Passed GetEmployees_Collection_AllItemsAreNotNull_Constraint 4 ms
Passed GetEmployees_Collection_ContainsEmployeeWithId100 6 ms Passed
GetEmployees_Collection_ContainsEmployeeWithId100_Constraint 5 ms
Passed GetEmployees_Collection_AllItemsAreUnique 5 ms Passed
GetEmployees_Collection_AllItemsAreUnique_Constraint 4 ms
Passed GetEmployees_AndPreviousYears_AreEqual 6 ms
Passed GetEmployees_AndPreviousYears_AreEquivalent 5 ms
Passed GetEmployees_AndPreviousYears_AreEqual_Constraint 5
ms
```

```
=====
Total: 9 Passed: 9 Failed: 0 Skipped: 0
Test Run Successful.
```

.